

AWS: Identity and Access Management

Identity and Access Management (IAM) ensures secure control over who can access cloud resources and what actions they are allowed to perform.

- Protects systems from unauthorized access and credential misuse
- Controls user identities, roles, and permissions centrally
- Follows the principle of least privilege for stronger security
- Reduces risks from weak, stolen, or shared passwords
- Essential for secure and scalable cloud environments

IAM Importance

- Weak, stolen, or default passwords are a major cause of data breaches, making identity protection the weakest link in cybersecurity.
- Attackers commonly use phishing to steal credentials, then escalate access to install malware, create backdoors, or steal sensitive data.
- IAM reduces this risk by enforcing controlled, auditable, and least-privilege access to resources.

Identity and Access Management (IAM)

IAM is a security framework that manages authentication (who can sign in) and authorization (what they are allowed to do) across systems and resources.

Amazon Web Services Identity and Access Management (IAM) is a web service that securely controls access to AWS resources and acts as the backbone of account-level security.

- Controls user identities and permissions centrally
- Prevents unauthorized access to cloud services
- Supports secure scaling of users, applications, and services

AWS Account Root User

The root user is the original identity created with an AWS account and has unrestricted access to all services and billing.

- Has full administrative privileges
- Should never be used for daily operations
- Must have Multi-Factor Authentication (MFA) enabled
- Reserved only for critical account-level tasks

Core Components of AWS IAM

IAM is built on several core components that represent "who" can access your account.

1. IAM Users

IAM Users represent individual people or applications requiring long-term access to AWS.

- Have permanent credentials (passwords and access keys)
- Used for console, CLI, or API access
- Best managed through groups, not individually

2. IAM Groups

IAM Groups are collections of users that simplify permission management.

- Permissions are assigned once at the group level
- Users inherit permissions automatically
- Reduces administrative errors and improves scalability

3. IAM Roles

IAM Roles provide temporary permissions to trusted entities without using long-term credentials.

- No passwords or access keys
- Uses temporary, auto-rotated credentials
- Ideal for AWS services and cross-account access
- Most secure method for application permissions

Identity and Access Management Working

Identity and Access Management (IAM) works by clearly defining who can access AWS, what they can access, and under what conditions. Its working can be understood in the following flow:

1. Identify the User or Entity (Who)

IAM first identifies who is requesting access. This could be:

- An IAM user (human)
- An application or service

- An AWS service assuming a role
Authentication is done using credentials such as passwords, access keys, or MFA.

2. Authenticate the Request

IAM verifies the identity using authentication mechanisms like:

- Username and password
- Access keys
- Multi-Factor Authentication (MFA)
Only authenticated identities can proceed further.

3. Authorize Using Policies (Can Access)

After authentication, IAM checks **policies** attached to the user, group, or role. These policies define:

- Allowed or denied actions (e.g., s3:GetObject)
- Specific resources (e.g., a particular S3 bucket)
- Conditions (time, IP address, MFA usage)

4. Evaluate Permissions

IAM evaluates permissions in this order:

- **Explicit Deny** → always blocks access
- **Explicit Allow** → grants access
- **Implicit Deny** → default if no allow exists

Access is granted only if an explicit allow exists and no deny overrides it.

5. Grant Access to Resources (What)

If authorization succeeds, IAM allows the user or service to perform the requested action on the specified AWS resource such as:

- EC2 instances
- S3 buckets
- RDS databases

6. Temporary Access via Roles (Optional but Recommended)

For applications and services, IAM roles provide temporary credentials instead of long-term keys, improving security and reducing credential misuse.

Access Management in IAM

Access management determines who can access which AWS resources and what actions they can perform.

- Based on the principle of least privilege
- Prevents over-permissioning
- Improves security posture and compliance

Federation and Single Sign-On (SSO)

Federation allows external users to access AWS using existing identity providers.

- Supports corporate directories and social logins
- Enables Single Sign-On (SSO)
- Reduces password fatigue and improves security

Role-Based Access Control (RBAC)

RBAC assigns permissions based on job roles rather than individual users.

- Simplifies access control

- Aligns permissions with organizational structure
- Minimizes security risks caused by excess privileges.

Types of Permissions in IAM

1. Allow Permissions

An explicit "allow" permission grants the user, group, or role access to a resource. For Example, an IAM user with the "s3:ListBucket" permission can list the content of an S3 Bucket.

2. Deny Permissions

A "deny" permission explicitly prevents access to a resource. This can be set for security reasons, such as denying access to sensitive resources for certain users.

3. Implicit Deny

By Default, if no policy is attached to a user, group, or role, access to a resource is denied. IAM denies access if no explicit allow rule is present.

Type	Description	Example
Allow	Grants access to perform specified actions.	s3:ListBucket allows listing bucket contents.
Explicit Deny	Overrides any allow rule to block access.	Deny access to production S3 buckets for non-admin roles.
Implicit Deny	Default state; if no policy explicitly allows access, it is denied.	A new user with no attached policies cannot access any resource.

How to Set up IAM in AWS: Step by Step Process

Setting up IAM involves creating users, groups, roles, and policies that define what resources and actions are allowed. Below is a simple process for getting started:

Step 1: Create an IAM Users

- Open the **IAM Console** and click **Users → Add User**.
- Enter the username, select the access type (e.g., programmatic or console access), and assign permissions.

Step 2: Create an IAM Group

- Go to **IAM Console >> Groups >> Create New Group**
- Add users to this group and assign a set of policies (permissions).

Step 3: Enable MFA (Multi-Factor Authentication)

- To enhance security, enable MFA for critical IAM users (such as admins). MFA adds an extra layer of protection by requiring users to provide a second form of authentication (e.g., a mobile app).

Step 4: Create IAM Roles

IAM roles can be created for cross-account access or to give AWS services permissions to interact with each other (E.g., EC2 needs access to S3).